МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЗАХІДНОУКРАЇНСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
Факультет комп’ютерних інформаційних технологій
Кафедра комп’ютерних наук

ЗВІТ

про виконання лабораторної роботи №2
з дисципліни «Методи аналізу інтервальних даних»
на тему: «Арифметичні операції з інтервальними матрицями»
Варіант 1

Виконав:
студент групи ІПЗМПм-11
Боднар А.

Перевірила:
к.т.н., доцент
Крепич С. Я.

Тернопіль – 2026

МЕТА РОБОТИ

Засвоїти, як виконуються арифметичні операції над матрицями, елементи яких є комплексними інтервалами; навчитися будувати такі матриці за заданими формулами та обчислювати матричний вираз з урахуванням правил інтервальної арифметики; реалізувати обчислення у вигляді програми з графічним інтерфейсом.

ЗАВДАННЯ

Обчислити інтервальний матричний вираз

$$Q = B \times C - A,$$

де A, B, C — квадратні матриці третього порядку, елементи яких є комплексними інтервалами з номерами від 1 до 27, розміщеними в матрицях по стовпцях (таблиця 1).

Таблиця 1 – Розміщення елементів у матрицях

Матриця A	**Матриця B**	**Матриця C**
1 4 7	10 13 16	19 22 25
2 5 8	11 14 17	20 23 26
3 6 9	12 15 18	21 24 27

Кожен елемент задається формулою, у яку підставляються особисті дані студента. Запис [x ± d] позначає інтервал [x − d; x + d].

ВИХІДНІ ДАНІ

Таблиця 2 – Дані студента

Параметр	**Позначення**	**Значення**
День народження	Д	12
Місяць народження	М	11
Остання цифра залікової книжки	ЗК	3
Остання цифра студентського квитка	СК	0
Порядковий номер у журналі	Ж	1
Порядковий номер на лабораторній	Л	2

ТЕОРЕТИЧНІ ВІДОМОСТІ

Інтервальна матриця — це матриця, кожен елемент якої є інтервалом. Її можна розглядати як множину всіх звичайних матриць, елементи яких належать відповідним інтервалам. Для інтервальних матриць зберігаються звичні означення операцій, але кожна дія над елементами виконується за правилами інтервальної арифметики.

Для дійсних інтервалів $A = [a_1; a_2]$ і $B = [b_1; b_2]$ використовуються формули:

$$A + B = [a_1 + b_1; a_2 + b_2], \quad A - B = [a_1 - b_2; a_2 - b_1],$$

$$A \cdot B = [\min\{a_1b_1, a_1b_2, a_2b_1, a_2b_2\}; \max\{a_1b_1, a_1b_2, a_2b_1, a_2b_2\}].$$

Комплексний інтервал записується у прямокутній формі $A = A_1 + iA_2$, де A_1 і A_2 — дійсні інтервали. Додавання і віднімання виконуються окремо для дійсних і уявних частин, а множення — за формулою

$$A \cdot B = (A_1B_1 - A_2B_2) + i(A_1B_2 + A_2B_1).$$

Різниця матриць однакового розміру обчислюється поелементно, а елемент добутку матриць — як сума добутків елементів рядка першої матриці на елементи стовпця другої:

$$(B \times C)_{ij} = B_{i1}C_{1j} + B_{i2}C_{2j} + B_{i3}C_{3j}.$$

Важливою особливістю інтервальної арифметики є те, що віднімання не є оберненим до додавання: $A - A \neq 0$ для невиродженого інтервалу. Тому кожна операція лише розширює результат, і ширина елементів Q швидко зростає.

ХІД РОБОТИ

1. Обчислення елементів матриць

Після підстановки даних з таблиці 2 у формули з умови отримано значення всіх 27 комплексних інтервалів (таблиця 3).

Таблиця 3 – Елементи матриць

№	Формула	Значення
1	[−1; 1] + i[СК ± 3]	[−1; 1] + i[−3; 3]
2	[Д ± 10] + i[−4; 0]	[2; 22] + i[−4; 0]

№	Формула	Значення
3	[−2; 2] + i[((Ж + Л) ± 4)/5]	[−2; 2] + i[−0.2; 1.4]
4	[М ± 5] + i[0; 1]	[6; 16] + i[0; 1]
5	[−3; 3] + i[((СК + 3К) ± 6)/4]	[−3; 3] + i[−0.75; 2.25]
6	[(Д + М)/2 ± 5] + i[−2; 2]	[6.5; 16.5] + i[−2; 2]
7	[−4; 4] + i[Д ± 10]	[−4; 4] + i[2; 22]
8	[((Д + М) ± 5)/4] + i[0; 3]	[4.5; 7] + i[0; 3]
9	[−5; 5] + i[Ж ± 2]	[−5; 5] + i[−1; 3]
10	[0; 1] + i[М ± 5]	[0; 1] + i[6; 16]
11	[3К ± 2] + i[−1; 0]	[1; 5] + i[−1; 0]
12	[0; 2] + i[(Ж + Л)/3 ± 4]	[0; 2] + i[−3; 5]
13	[СК ± 3] + i[0; 5]	[−3; 3] + i[0; 5]
14	[0; 3] + i[(Д + М)/2 ± 5]	[0; 3] + i[6.5; 16.5]
15	[(СК + 3К)/2 ± 4] + i[−1; 1]	[−2.5; 5.5] + i[−1; 1]
16	[0; 4] + i[Л ± 5]	[0; 4] + i[−3; 7]
17	[((СК + 3К) ± 6)/4] + i[−4; 4]	[−0.75; 2.25] + i[−4; 4]
18	[0; 5] + i[((Д + М) ± 5)/4]	[0; 5] + i[4.5; 7]
19	[−5; 0] + i[(СК + 3К)/2 ± 4]	[−5; 0] + i[−2.5; 5.5]
20	[Ж ± 2] + i[0; 2]	[−1; 3] + i[0; 2]
21	[−4; 0] + i[3К ± 2]	[−4; 0] + i[1; 5]
22	[Л ± 5] + i[−3; 3]	[−3; 7] + i[−3; 3]
23	[−3; 0] + i[−5; 0]	[−3; 0] + i[−5; 0]
24	[(Ж + Л)/3 ± 4] + i[−2; 0]	[−3; 5] + i[−2; 0]
25	[−2; 0] + i[−5; 5]	[−2; 0] + i[−5; 5]
26	[((Ж + Л) ± 4)/5] + i[0; 4]	[−0.2; 1.4] + i[0; 4]
27	[−1; 0] + i[−3; 0]	[−1; 0] + i[−3; 0]

2. Формування матриць A, B, C

Розміщенням елементів за схемою з таблиці 1 сформовано матриці A, B і C (таблиці 4–6). У кожній клітинці вказано номер елемента, дійсну (Re) та уявну (Im) частини.

Таблиця 4 – Матриця A

A	№1 Re [−1; 1] Im [−3; 3]	№4 Re [6; 16] Im [0; 1]	№7 Re [−4; 4] Im [2; 22]
	№2 Re [2; 22] Im [−4; 0]	№5 Re [−3; 3] Im [−0.75; 2.25]	№8 Re [4.5; 7] Im [0; 3]
	№3 Re [−2; 2] Im [−0.2; 1.4]	№6 Re [6.5; 16.5] Im [−2; 2]	№9 Re [−5; 5] Im [−1; 3]

Таблиця 5 – Матриця B

B	№10 Re [0; 1] Im [6; 16]	№13 Re [−3; 3] Im [0; 5]	№16 Re [0; 4] Im [−3; 7]
	№11 Re [1; 5] Im [−1; 0]	№14 Re [0; 3] Im [6.5; 16.5]	№17 Re [−0.75; 2.25] Im [−4; 4]
	№12 Re [0; 2] Im [−3; 5]	№15 Re [−2.5; 5.5] Im [−1; 1]	№18 Re [0; 5] Im [4.5; 7]

Таблиця 6 – Матриця C

C	№19 Re [−5; 0] Im [−2.5; 5.5]	№22 Re [−3; 7] Im [−3; 3]	№25 Re [−2; 0] Im [−5; 5]
	№20 Re [−1; 3] Im [0; 2]	№23 Re [−3; 0] Im [−5; 0]	№26 Re [−0.2; 1.4] Im [0; 4]
	№21 Re [−4; 0] Im [1; 5]	№24 Re [−3; 5] Im [−2; 0]	№27 Re [−1; 0] Im [−3; 0]

3. Обчислення елемента Q_{11} вручну

Для перевірки роботи програми один елемент результуючої матриці обчислено вручну:

$$Q_{11} = B_{11}\cdot C_{11} + B_{12}\cdot C_{21} + B_{13}\cdot C_{31} - A_{11}.$$

Добуток дійсних інтервалів визначається як мінімум і максимум чотирьох добутків їхніх кінців, наприклад: [0; 1]·[−5; 0] = [min{0, 0, −5, 0}; max{…}] = [−5; 0].

$$B_{11}\cdot C_{11} = ([0; 1] + i[6; 16])\cdot([-5; 0] + i[-2.5; 5.5])$$

$$\text{Re} = [0; 1]\cdot[-5; 0] - [6; 16]\cdot[-2.5; 5.5] = [-5; 0] - [-40; 88] = [-93; 40]$$

$$\text{Im} = [0; 1]\cdot[-2.5; 5.5] + [6; 16]\cdot[-5; 0] = [-2.5; 5.5] + [-80; 0] = [-82.5; 5.5]$$

$$B_{12}\cdot C_{21} = ([-3; 3] + i[0; 5])\cdot([-1; 3] + i[0; 2])$$

$$\text{Re} = [-3; 3]\cdot[-1; 3] - [0; 5]\cdot[0; 2] = [-9; 9] - [0; 10] = [-19; 9]$$

$$\text{Im} = [-3; 3]\cdot[0; 2] + [0; 5]\cdot[-1; 3] = [-6; 6] + [-5; 15] = [-11; 21]$$

$$B_{13}\cdot C_{31} = ([0; 4] + i[-3; 7])\cdot([-4; 0] + i[1; 5])$$

$$\text{Re} = [0; 4]\cdot[-4; 0] - [-3; 7]\cdot[1; 5] = [-16; 0] - [-15; 35] = [-51; 15]$$

$$\mathrm{Im} = [0; 4]\cdot[1; 5] + [-3; 7]\cdot[-4; 0] = [0; 20] + [-28; 12] = [-28; 32]$$

Сума трьох добутків:

$$(B \times C)_{11} = ([-93; 40] + i[-82.5; 5.5]) + ([-19; 9] + i[-11; 21]) + ([-51; 15] + i[-28; 32]) = [-163; 64] + i[-121.5; 58.5].$$

Після віднімання елемента A_{11}:

$$Q_{11} = ([-163; 64] + i[-121.5; 58.5]) - ([-1; 1] + i[-3; 3]) = [-164; 65] + i[-124.5; 61.5].$$

Результат збігається зі значенням, яке показує програма (рисунок 3). Решта елементів обчислюється так само; у програмі розпис будь-якого елемента відкривається натисканням на нього.

4. Результуюча матриця

Результати множення B × C та віднімання матриці A наведено в таблицях 7 і 8.

Таблиця 7 – Матриця B × C

B×C	Re [−163; 64] Im [−121.5; 58.5]	Re [−78; 123] Im [−110; 165]	Re [−119.2; 105.2] Im [−69; 27]
	Re [−92.5; 37.5] Im [−48.75; 115.25]	Re [−41.75; 139.75] Im [−111; 39.5]	Re [−95.85; 21.95] Im [−39.05; 68.35]
	Re [−102; 30.5] Im [−66; 65]	Re [−57.5; 80.5] Im [−88.5; 91.5]	Re [−41.5; 57.7] Im [−53.4; 39.4]

Таблиця 8 – Результуюча матриця Q = B × C − A

Q	Re [−164; 65] Im [−124.5; 61.5]	Re [−94; 117] Im [−111; 165]	Re [−123.2; 109.2] Im [−91; 25]
	Re [−114.5; 35.5] Im [−48.75; 119.25]	Re [−44.75; 142.75] Im [−113.25; 40.25]	Re [−102.85; 17.45] Im [−42.05; 68.35]
	Re [−104; 32.5] Im [−67.4; 65.2]	Re [−74; 74] Im [−90.5; 93.5]	Re [−46.5; 62.7] Im [−56.4; 40.4]

5. Програмна реалізація

Програму реалізовано як вебзастосунок мовою TypeScript з бібліотеками React і React-Bootstrap; він працює в браузері й розміщений на GitHub Pages (посилання наведено нижче). Обчислювальна частина відокремлена від інтерфейсу і не залежить від нього:

– interval.ts — клас Interval (дійсний інтервал) з операціями додавання, віднімання, множення і ділення;
– complex.ts — клас ComplexInterval (комплексний інтервал у прямокутній формі);
– matrix.ts — додавання, віднімання і множення матриць комплексних інтервалів;
– lab2.ts — формули 27 елементів, схема їх розміщення в матрицях, перевірка вхідних даних і функція solveLab2, що повертає A, B, C, B × C та Q;
– format.ts — виведення меж інтервалів з округленням назовні, щоб показаний інтервал гарантовано містив точний.

Інтерфейс складається з трьох послідовних кроків: введення даних, перегляд сформованих матриць і результат. Програма перевіряє введені значення (цілі числа в допустимих межах, існування дати народження) і не дає перейти до обчислень, доки в полях є помилки. Правильність обчислювальної частини перевірено автоматичними тестами (Vitest), зокрема на прикладах з лекцій.

Сторінка роботи на сайті: https://4ndruxa.github.io/university-labs/#/maid/lab2.

Репозиторій з кодом: https://github.com/4ndruxa/university-labs.

6. Результати роботи програми

На рисунку 1 наведено вікно введення початкових даних.

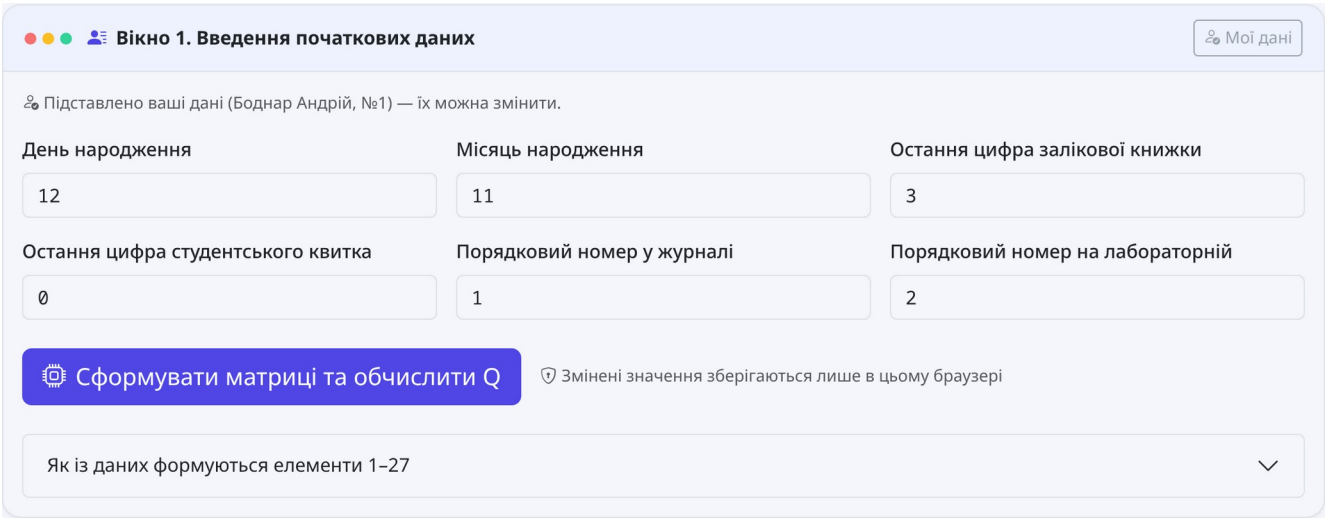


Рисунок 1 – Вікно введення початкових даних

На рисунку 2 наведено вікно зі сформованими матрицями A, B і C.



Рисунок 2 – Вікно зі сформованими матрицями A, B, C

На рисунку 3 наведено вікно з результуючою матрицею Q і розписом елемента Q_{11}.

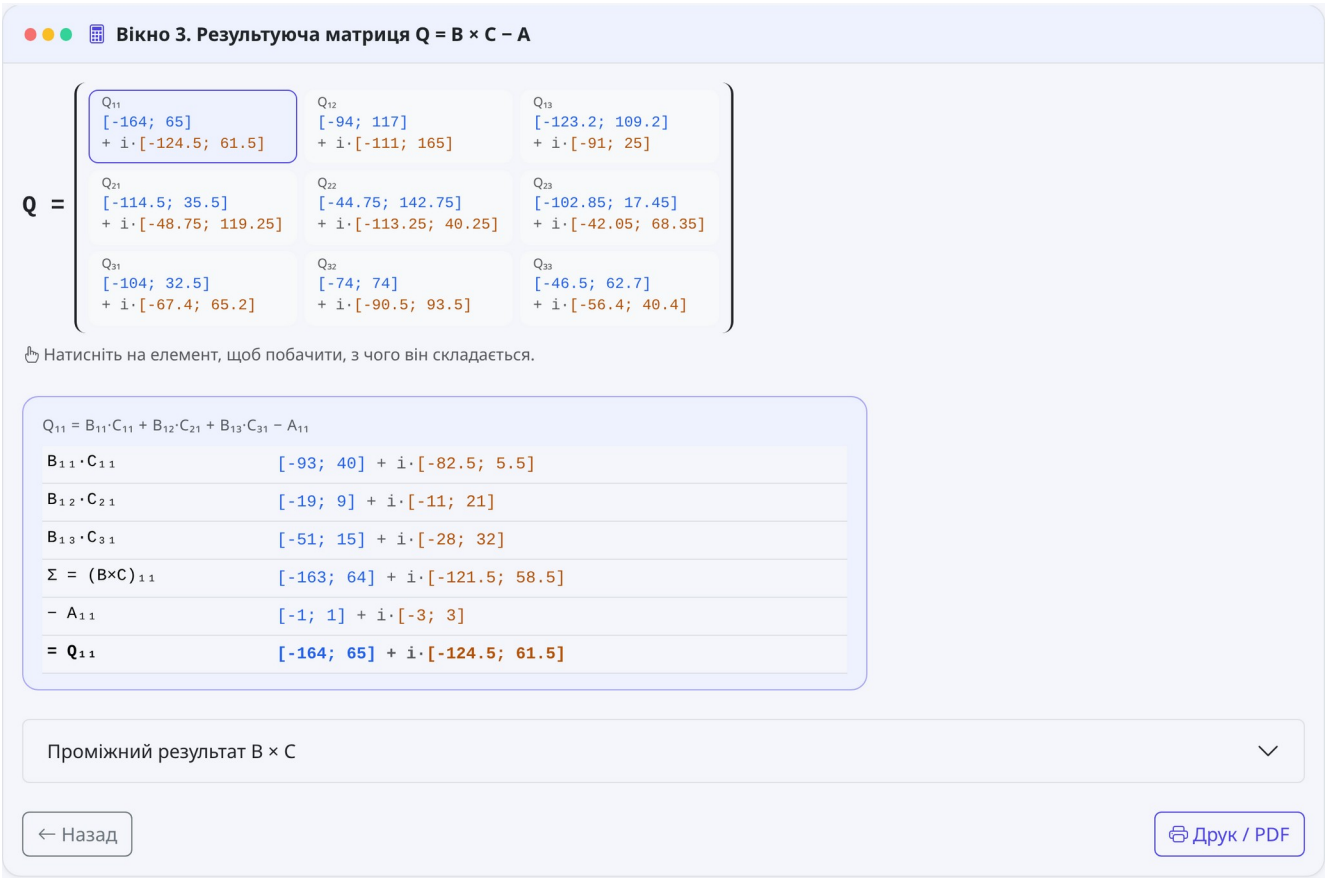


Рисунок 3 – Вікно з результуючою матрицею Q

ВИСНОВКИ

У ході виконання лабораторної роботи опрацьовано операції над матрицями комплексних інтервалів і розроблено вебзастосунок, який за особистими даними студента формує матриці A, B, C та обчислює вираз Q = B × C − A.

Елементи отриманої матриці значно ширші за вихідні: наприклад, Q_{11} = [−164; 65] + i[−124.5; 61.5], а найширшим є Q_{12} = [−94; 117] + i[−111; 165]. Розширення пояснюється тим, що кожен елемент добутку містить шість множень і п’ять додавань і віднімань інтервалів, а жодна інтервальна операція не звужує результат. Найсильніше розширюються елементи, у яких множаться інтервали, що містять нуль або мають велику ширину.

Ручне обчислення елемента Q_{11} дало той самий результат, що й програма, отже, реалізація формул правильна. Застосунок дозволяє змінити вхідні дані та одразу побачити нові матриці й розпис будь-якого елемента результату.

ДОДАТОК А. ЛІСТИНГ ПРОГРАМНОГО КОДУ

Нижче наведено обчислювальну частину програми. Повний код застосунку, разом з інтерфейсом і тестами, розміщено в репозиторії https://github.com/4ndruxa/university-labs.

Файл lab2.ts

```
// ЛР2: Q = B × C − A, елементи матриць 1…27 — за формулами з умови
import { ComplexInterval } from './complex'
import { Interval } from './interval'
import { matMul, matSub, type Matrix } from './matrix'

export interface StudentData {
  day: number // день народження
  month: number // місяць народження
  zal: number // остання цифра заліковоі книжки
  stud: number // остання цифра студентського квитка
  jrn: number // порядковий номер у журналі
  lab: number // порядковий номер на лабораторній
}

export const FIELD_LABELS: Record<keyof StudentData, string> = {
  day: 'День народження',
  month: 'Місяць народження',
  zal: 'Остання цифра заліковоі книжки',
  stud: 'Остання цифра студентського квитка',
  jrn: 'Порядковий номер у журналі',
  lab: 'Порядковий номер на лабораторній',
}

/** [c ± d] = [c − d; c + d] */
const pm = (c: number, d: number): Interval => new Interval(c - d, c + d)
const I = (a: number, b: number): Interval => new Interval(a, b)
const C = (re: Interval, im: Interval): ComplexInterval => new ComplexInterval(re,
    im)

/** Індекс 0 не використовується */
export function buildElements(d: StudentData): ComplexInterval[] {
  const dm = d.day + d.month
  const sz = d.stud + d.zal
  const jl = d.jrn + d.lab
  return [
    C(I(0, 0), I(0, 0)),
    C(I(-1, 1), pm(d.stud, 3)), // 1
    C(pm(d.day, 10), I(-4, 0)), // 2
    C(I(-2, 2), I((jl - 4) / 5, (jl + 4) / 5)), // 3
    C(pm(d.month, 5), I(0, 1)), // 4
    C(I(-3, 3), I((sz - 6) / 4, (sz + 6) / 4)), // 5
    C(pm(dm / 2, 5), I(-2, 2)), // 6
    C(I(-4, 4), pm(d.day, 10)), // 7
    C(I((dm - 5) / 4, (dm + 5) / 4), I(0, 3)), // 8
    C(I(-5, 5), pm(d.jrn, 2)), // 9
    C(I(0, 1), pm(d.month, 5)), // 10
    C(pm(d.zal, 2), I(-1, 0)), // 11
    C(I(0, 2), pm(jl / 3, 4)), // 12
    C(pm(d.stud, 3), I(0, 5)), // 13
```

```
    C(I(0, 3), pm(dm / 2, 5)), // 14
    C(pm(sz / 2, 4), I(-1, 1)), // 15
    C(I(0, 4), pm(d.lab, 5)), // 16
    C(I((sz - 6) / 4, (sz + 6) / 4), I(-4, 4)), // 17
    C(I(0, 5), I((dm - 5) / 4, (dm + 5) / 4)), // 18
    C(I(-5, 0), pm(sz / 2, 4)), // 19
    C(pm(d.jrn, 2), I(0, 2)), // 20
    C(I(-4, 0), pm(d.zal, 2)), // 21
    C(pm(d.lab, 5), I(-3, 3)), // 22
    C(I(-3, 0), I(-5, 0)), // 23
    C(pm(jl / 3, 4), I(-2, 0)), // 24
    C(I(-2, 0), I(-5, 5)), // 25
    C(I((jl - 4) / 5, (jl + 4) / 5), I(0, 4)), // 26
    C(I(-1, 0), I(-3, 0)), // 27
  ]
}

/** Номери елементів у матрицях — по стовпцях */
export const LAYOUT = {
  A: [[1, 4, 7], [2, 5, 8], [3, 6, 9]],
  B: [[10, 13, 16], [11, 14, 17], [12, 15, 18]],
  C: [[19, 22, 25], [20, 23, 26], [21, 24, 27]],
} as const


// Рік невідомий, тому 29.02 допустиме
const DAYS_IN_MONTH = [31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
const MONTH_IN = ['січні', 'лютому', 'березні', 'квітні', 'травні', 'червні',
    'липні', 'серпні', 'вересні', 'жовтні', 'листопаді', 'грудні']


/** Лише звичайне ціле число (без '1e1', '-0', '0x10', '2.5'); інакше NaN */
export function parseField(s: string): number {
  const t = s.trim()
  return /^\d{1,3}$/.test(t) ? Number(t) : Number.NaN
}

/** Порожній об'єкт — дані коректні */
export function validate(d: StudentData): Partial<Record<keyof StudentData,
    string>> {
  const errors: Partial<Record<keyof StudentData, string>> = {}
  const check = (k: keyof StudentData, lo: number, hi: number, message: string) =>
    {
    const v = d[k]
    if (!Number.isInteger(v) || v < lo || v > hi) errors[k] = message
  }
  check('day', 1, 31, 'День — від 1 до 31')
  check('month', 1, 12, 'Місяць — від 1 до 12')
  check('zal', 0, 9, 'Одна цифра, 0–9')
  check('stud', 0, 9, 'Одна цифра, 0–9')
  check('jrn', 1, 100, 'Число від 1 до 100')
  check('lab', 1, 100, 'Число від 1 до 100')
  if (!errors.day && !errors.month && d.day > DAYS_IN_MONTH[d.month - 1]) {
    errors.day = `У ${MONTH_IN[d.month - 1]} ${d.month === 2 ? 'не більше 29' :
    DAYS_IN_MONTH[d.month - 1]} днів`
  }
  return errors
}

export interface Lab2Result {
  A: Matrix
  B: Matrix
  C: Matrix
  BC: Matrix
```

```
  Q: Matrix
}

export function solveLab2(d: StudentData): Lab2Result {
  const errors = validate(d)
  const keys = Object.keys(errors) as (keyof StudentData)[]
  if (keys.length) throw new Error(keys.map((k) => `${FIELD_LABELS[k]}: $
    {errors[k]}`).join('\n'))
  const e = buildElements(d)
  const place = (layout: readonly (readonly number[])[]): Matrix =>
    layout.map((row) => row.map((n) => e[n]))
  const A = place(LAYOUT.A)
  const B = place(LAYOUT.B)
  const C = place(LAYOUT.C)
  const BC = matMul(B, C)
  return { A, B, C, BC, Q: matSub(BC, A) }
}
```

Файл matrix.ts

```
// Матриці комплексних інтервалів: (A × B)ij = Σk Aik · Bkj
import { ComplexInterval } from './complex'

export type Matrix = ComplexInterval[][]

function size(m: Matrix): [number, number] {
  if (m.length === 0 || m[0].length === 0) throw new Error('Матриця порожня')
  return [m.length, m[0].length]
}

function checkSameSize(a: Matrix, b: Matrix): void {
  const [ra, ca] = size(a)
  const [rb, cb] = size(b)
  if (ra !== rb || ca !== cb) throw new Error(`Матриці мають різні розміри: ${ra}
    ×${ca} і ${rb}×${cb}`)
}

export function matAdd(a: Matrix, b: Matrix): Matrix {
  checkSameSize(a, b)
  return a.map((row, i) => row.map((x, j) => x.add(b[i][j])))
}

export function matSub(a: Matrix, b: Matrix): Matrix {
  checkSameSize(a, b)
  return a.map((row, i) => row.map((x, j) => x.sub(b[i][j])))
}

/** Доданки Aik · Bkj елемента (i, j) */
export function productTerms(a: Matrix, b: Matrix, i: number, j: number):
    ComplexInterval[] {
  return a[i].map((aik, k) => aik.mul(b[k][j]))
}

export function matMul(a: Matrix, b: Matrix): Matrix {
  const [ra, ca] = size(a)
  const [rb, cb] = size(b)
  if (ca !== rb) throw new Error(`Не можна помножити матриці ${ra}×${ca} і ${rb}×$
    {cb}`)
  return a.map((_, i) =>
    Array.from({ length: cb }, (_, j) =>
```

```
      productTerms(a, b, i, j).reduce((sum, t) => sum.add(t),
    ComplexInterval.ZERO)))
}
```

Файл complex.ts

```
// Комплексний інтервал у прямокутній формі A = A1 + i·A2
import { Interval, type IntervalLike, type Warn } from './interval'

export class ComplexInterval {
  readonly re: Interval
  readonly im: Interval

  constructor(re: IntervalLike, im: IntervalLike) {
    this.re = Interval.from(re)
    this.im = Interval.from(im)
  }

  static readonly ZERO = new ComplexInterval(0, 0)

  /** A + B = (A1 + B1) + i(A2 + B2) */
  add(b: ComplexInterval): ComplexInterval {
    return new ComplexInterval(this.re.add(b.re), this.im.add(b.im))
  }

  /** A − B = (A1 − B1) + i(A2 − B2) */
  sub(b: ComplexInterval): ComplexInterval {
    return new ComplexInterval(this.re.sub(b.re), this.im.sub(b.im))
  }

  /** A · B = (A1B1 − A2B2) + i(A1B2 + A2B1) */
  mul(b: ComplexInterval): ComplexInterval {
    return new ComplexInterval(
      this.re.mul(b.re).sub(this.im.mul(b.im)),
      this.re.mul(b.im).add(this.im.mul(b.re)),
    )
  }

  /** A ÷ B = (A1B1 + A2B2)/(B1² + B2²) + i(A2B1 − A1B2)/(B1² + B2²) */
  div(b: ComplexInterval, warn?: Warn): ComplexInterval {
    const den = b.re.sqr().add(b.im.sqr())
    return new ComplexInterval(
      this.re.mul(b.re).add(this.im.mul(b.im)).div(den, warn),
      this.im.mul(b.re).sub(this.re.mul(b.im)).div(den, warn),
    )
  }

  equals(b: ComplexInterval, eps = 1e-9): boolean {
    return this.re.equals(b.re, eps) && this.im.equals(b.im, eps)
  }

  toString(digits?: number): string {
    return `${this.re.toString(digits)} + i·${this.im.toString(digits)}`
  }
}
```

Файл interval.ts

```
// Дійсний інтервал [a1; a2]
```

```
import { fmtHi, fmtLo } from './format'

export type Warn = (message: string) => void
export type IntervalLike = Interval | readonly [number, number] | number

export class Interval {
  readonly lo: number
  readonly hi: number

  constructor(lo: number, hi: number = lo) {
    if (!Number.isFinite(lo) || !Number.isFinite(hi)) throw new Error('Межі
    інтервалу мають бути скінченними числами')
    if (lo > hi) throw new Error(`Некоректний інтервал [${lo}; ${hi}]: ліва межа
    більша за праву`)
    this.lo = lo + 0 // −0 → 0
    this.hi = hi + 0
  }

  static from(x: IntervalLike): Interval {
    if (x instanceof Interval) return x
    if (typeof x === 'number') return new Interval(x, x)
    return new Interval(x[0], x[1])
  }

  /** A + B = [a1 + b1; a2 + b2] */
  add(other: IntervalLike): Interval {
    const b = Interval.from(other)
    return new Interval(this.lo + b.lo, this.hi + b.hi)
  }

  /** A − B = [a1 − b2; a2 − b1] */
  sub(other: IntervalLike): Interval {
    const b = Interval.from(other)
    return new Interval(this.lo - b.hi, this.hi - b.lo)
  }

  /** Чотири добутки кінців {a1b1, a1b2, a2b1, a2b2} */
  products(other: IntervalLike): number[] {
    const b = Interval.from(other)
    return [this.lo * b.lo, this.lo * b.hi, this.hi * b.lo, this.hi * b.hi]
  }

  /** Чотири частки кінців {a1/b1, a1/b2, a2/b1, a2/b2} */
  quotients(other: IntervalLike): number[] {
    const b = Interval.from(other)
    return [this.lo / b.lo, this.lo / b.hi, this.hi / b.lo, this.hi / b.hi]
  }

  /** A · B = [min{a1b1, a1b2, a2b1, a2b2}; max{…}] */
  mul(other: IntervalLike): Interval {
    const p = this.products(other)
    return new Interval(Math.min(...p), Math.max(...p))
  }

  /**
   * A / B = [min{a1/b1, a1/b2, a2/b1, a2/b2}; max{…}]
   * 0 усередині B: те саме правило, що в лекції (приклад 3), з попередженням
    через warn.
   */
  div(other: IntervalLike, warn?: Warn): Interval {
```

```
    const b = Interval.from(other)
    if (b.lo === 0 || b.hi === 0) throw new Error(`Ділення на ${b} неможливе: межа
    дільника дорівнює 0`)
    if (b.lo < 0 && b.hi > 0) warn?.(`дільник ${b} містить 0 — застосовано правило
    min/max чотирьох часток (лекція 1, приклад 3)`)
    const q = this.quotients(b)
    return new Interval(Math.min(...q), Math.max(...q))
  }

  /** −A = [−a2; −a1] */
  neg(): Interval {
    return new Interval(-this.hi, -this.lo)
  }

  /** {x² | x ∈ A} */
  sqr(): Interval {
    const a = this.lo * this.lo
    const b = this.hi * this.hi
    return this.contains(0) ? new Interval(0, Math.max(a, b)) : new
    Interval(Math.min(a, b), Math.max(a, b))
  }

  contains(x: number): boolean {
    return this.lo <= x && x <= this.hi
  }

  /** wid(A) = a2 − a1 */
  get width(): number {
    return this.hi - this.lo
  }

  /** mid(A) = (a1 + a2) / 2 */
  get mid(): number {
    return (this.lo + this.hi) / 2
  }

  equals(other: IntervalLike, eps = 1e-9): boolean {
    const b = Interval.from(other)
    return Math.abs(this.lo - b.lo) <= eps && Math.abs(this.hi - b.hi) <= eps
  }

  toString(digits?: number): string {
    return `[${fmtLo(this.lo, digits)}; ${fmtHi(this.hi, digits)}]`
  }
}
```

Файл format.ts

```
// Межі округлюються назовні (ліва вниз, права вгору), щоб показаний інтервал
    містив точний

export const DIGITS = 4

const noMinusZero = (x: number): number => (Object.is(x, -0) ? 0 : x)

function roundDir(x: number, digits: number, dir: 'down' | 'up'): number {
  const k = 10 ** digits
  const scaled = x * k
  if (!Number.isFinite(scaled)) return x
```

```
  const nearest = Math.round(scaled)
  // Не розширюємо інтервал через похибку float (0.1 + 0.2)
  if (Math.abs(scaled - nearest) < 1e-9 * Math.max(1, Math.abs(scaled))) return
    noMinusZero(nearest / k)
  return noMinusZero((dir === 'down' ? Math.floor(scaled) : Math.ceil(scaled)) /
    k)
}

export const fmtLo = (x: number, digits = DIGITS): string => String(roundDir(x,
    digits, 'down'))
export const fmtHi = (x: number, digits = DIGITS): string => String(roundDir(x,
    digits, 'up'))
export const fmtNum = (x: number, digits = DIGITS): string =>
    String(noMinusZero(Number(x.toFixed(digits))))
```